

COLX 585: Membership Inference Attack in VLMs Project: Milestone 6

Yucai Zhong and Christina McCallum and Jennifer Flake and Rachelle De Jager

Abstract

For this milestone, we implemented three new models to perform Membership Inference Attack (MIA) in Vision-Language Models (VLMs). The methods of attack include a model that uses a filtered feature approach (using existing features), a model that uses additional neighbor features with neural stacking and routed blend, and, lastly, a model that implements PC-MMIA features, in addition to lightweight feature tweaking. The highest gains overall were achieved by the filtered feature approach, while the PC-MMIA scored highest on the test set.

1 Introduction

After multiple attempts at MIA in VLMs, we used this milestone to filter features from previous models and test new features on new models. The first model revisits the features that have been engineered so far and thoughtfully removes features that have a negative effect on task performance. The second model uses the first model as a starting point and then adds neighbor-based features to the model. Next, the second model uses both a routed blend and neural stacking in place of simple ensembling. Finally, the third model continues the work of feature selection started by the first model. In addition to tweaking features, this model adds PC-MMIA features, which is a perturbation-based calibration method that measures the difference in token log-probability.

The only significant gains in this milestone were from the first model's lightweight pipeline implementation, which filtered features methodically. This result demonstrates that more features may confuse or decrease the learning signal of the model. Combining features thoughtfully and systematically can improve a model's performance on the task.

2 Background

Summary of related research papers:

2.1 Paper 1: "LUMIA: Linear probing for Unimodal and MultiModal Membership Inference Attacks leveraging internal LLM States" (Ibanez-Lissen et al. 2025)

This paper extends the Membership Inference Attack (MIA) task by using Linear Probes (LPs) to examine the internal activations of LLM layers and determine if this information can be used to expose model vulnerabilities of membership data. Specifically, the authors explore unimodal (text only) and multimodal (visual and visual + text) paradigms across diverse datasets, and propose LUMIA (Linear probe-based Utilization of Model Internal Activations) as a pipeline of linear probe training. LUMIA uses a pre-trained model and a set of labeled samples where some are members and some are not, feeds the samples to the model, and finds the internal activations of each transformer layer. For each layer, an LP (in this case, a multilayer perceptron) is trained to predict whether a sample is a member or not based on the activation layer vector. AUC is outputted at each layer.

In this summary, we focus on the multimodal paradigm for its relevance to our task. The researchers use the Llava-OneVision model (on a variety of model sizes) and the datasets Textcaps, MathV360k, AOK, ChartQA, ScienceQA, IconQA, and Magpie, which are all drawn from the OneVision-Data collection. 85.9% of their experiments achieve an AUC greater than 0.6 (there is no previous work from which to compare this result, so this is the main metric). The paper's main findings suggest that datasets with more varied images and texts are more vulnerable to MIA (compared to uniform datasets), and that in the visual and visual + text modalities, layers 12-13 exhibit a spike in information used for MIA, with deeper layers

(around layer 15) adding more information. The multimodal case also showed that larger models tend to reveal more information in both the textual and visual parts of the models.

2.2 Paper 2: "Self-Comparison for Dataset-Level Membership Inference in Large (Vision-)Language Models" (Ren et al. 2025)

The methods of this paper explore MIA techniques in which paraphrasing determines membership data: the model will be agnostic to non-member paraphrasing, while it will assign higher likelihood to paraphrased member data, as it will have seen the original source. The authors evaluate this data-set level inference task with LLMs and LVMs and apply their methods to well-known API-based models. They prove that their Self-Comparison Membership Inference (SMI) performs better than existing techniques without the need for white-box access to model parameters. The LLMs used in this study are Pythia and GPT-Neo with the PILE training dataset, with New Yorker Caption Contest and FineWeb as non-member and auxiliary datasets. The VLMs used are LLaVa and CogVLM with the TextVQA, Visual Genome, with MS COCO training data for the former, and CogVLM-SFT for the latter. The non-member and auxiliary datasets are NoCaps and Flickr. They test on fine-tuned models Pythia-1.4B and LLaVA, and on the API-based model GPT-4o.

SMI involves measuring how confident a model predicts a particular text by assigning probability values to a sequence of tokens (using average negative log-likelihood). They find that after paraphrasing the second half of a sequence, the average negative log-likelihood of the member dataset increases with significance compared to non-members. They analyze the p-value of the hypothesis test between datasets before and after paraphrasing for their "self-comparison" method, and find the p-value to decrease rapidly across sample sizes for membership data.

3 Milestone 6

3.1 Method 1: Lightweight Pipeline

Compared with last week's implementation, the main change in this model was not the introduction of a completely new attack objective, but a more systematic use of the signals already extracted in Milestone 5. We merged three feature families, namely temperature-based features, Rényi-entropy-

based features, and image-perturbation features, into a unified pipeline. We then added several lightweight structural features derived from sample IDs, including source-related indicators, tokenized ID components, ID length, and numeric suffix patterns. These additional features were intended to capture dataset structure that was not explicitly represented in the original feature files.

After constructing the full feature space, we retained the strongest features, reduced redundancy among highly correlated variables, and evaluated several lightweight classifiers. In particular, we tested a logistic regression baseline, a global XGBoost classifier, a source-routed XGBoost variant, and a weighted blend of the different prediction streams. These methods differ in both model capacity and how they use the available information. Logistic regression provides a linear baseline over the merged feature set. Global XGBoost captures non-linear interactions across all feature families. The source-routed model assumes that different ID sources may follow different membership patterns and therefore trains specialized predictors for different groups. The weighted blend attempts to combine the strengths of the linear, global non-linear, and routed models. Relative to last week, the main improvement therefore came from organizing the existing information more effectively, exploiting simple dataset structure, and using a stronger modeling framework rather than introducing a completely new feature family.

The implementation of this method is provided in `lightweight_pipeline.py` in the `milestone6` folder. The script loads the three feature families from `data/extracted_features`, merges them by sample ID, adds ID-based structural features, trains all model variants, evaluates them on the validation split, and writes predictions and metrics to the output directory. The current implementation reports validation AUC directly in the script. In addition, we computed $\text{TPR@FPR} = 0.1$ from the validation predictions using the ROC curve, so that the evaluation reflects both overall ranking quality and performance at a low false-positive operating point.

Table 1 summarizes the validation results of the main variants. Among the combined models, the global XGBoost classifier achieved the best validation AUC, while the Rényi-only XGBoost baseline was the strongest single-feature-family model on this split.

Table 1: Method 1: Lightweight Pipeline

Method	AUC	TPR@FPR=0.1
Temperature Features + XGBoost	0.8495	0.3600
Rényi Features + XGBoost	0.8533	0.3400
Image Perturbation Features + XGBoost	0.8455	0.3467
Logistic Regression	0.8319	0.2467
Global XGBoost	0.8514	0.3133
Source-Routed XGBoost	0.8427	0.3133
Weighted Blend	0.8330	0.2633

This method works by aggregating complementary membership signals from multiple feature families into a single attack pipeline. The temperature-based and Rényi-based features characterize uncertainty, confidence concentration, and distributional sharpness in the model outputs, while the image-perturbation features capture robustness under visual changes. Logistic regression serves as a simple baseline, whereas XGBoost is better suited to exploiting non-linear interactions among these heterogeneous features. The routed variant further examines whether source-specific specialization improves membership prediction.

The main advantage of this approach is that it is simple, computationally efficient, and easy to maintain as a single Python script, while still achieving relatively strong validation performance. It also makes effective use of the features already extracted in the previous milestone, so the gain mainly came from combining and organizing existing information more carefully. However, the results also show that the more complex routing and blending strategies did not outperform the strongest global XGBoost baseline. This suggests that the routed models may overfit source-specific patterns, or that the current blending strategy does not calibrate the component models effectively. Possible future improvements include replacing validation-based feature selection with fold-based selection, using out-of-fold stacking instead of a manually weighted blend, and incorporating stronger raw-text or raw-image features to complement the current feature set.

3.2 Method 2: Neural Stacking Model and Neighbor Features

While we read "Membership Inference Attacks against Language Models via Neighbourhood Comparison" (Mattern et al., 2023) during milestone 1, we thought that revisiting the neighbor method

of attack for this milestone could be worthwhile. Our neighbor-based features are closely related to the neighborhood comparison approach proposed by Mattern et al. (2023), which evaluates a sample by comparing its likelihood to that of synthetically generated neighboring texts. We added these neighbor features to this milestone’s method 1: Lightweight Pipeline to see if these features would improve our model.

For this implementation, we used alternative captions drawn from the existing data examples as neighbors for the same image and prompt. For each of these neighbor captions, the model computes a caption-only cross-entropy loss by masking out prompt and padding tokens, ensuring that only the likelihood of the caption itself is evaluated. This produces a set of loss values that reflect the confidence that the model has for each alternative caption to match the image. As in neighborhood-based membership inference attacks, the signal comes from relative comparison: training examples tend to receive substantially lower loss than their neighbors, indicating that the model assigns them higher likelihood compared to similar alternatives. Yet, for non-training examples, the loss differences between the true caption and its neighbors are typically smaller, reflecting less confidence. By using real captions rather than synthetic neighbors, this approach still captures the same core idea of comparing local likelihoods to detect memorization effects. From these per-neighbor losses, summary statistics such as the mean, minimum, variance, and the gap between the true caption loss and the lowest (or average) neighbor loss are computed. These statistics were meant to capture how strongly the model prefers the observed caption over alternatives found in the same dataset.

In addition to adding neighbor features, this model adds neural stacking as a method of learned ensembling. A neural network replaces fixed averaging by learning how to combine the predictions of logistic regression and XGBoost. The pipeline begins similarly to previous models with feature engineering and includes new neighbor features. After that, logistic regression and XGBoost are used as independent classifiers, similar to how we have used them in the past. The outputs of logistic regression and XGBoost are used as inputs to a small multilayer perceptron (MLP). The MLP trained on this stacked representation to produce the final prediction. The MLP is made up of two hidden layers with ReLU activations and dropout. Neural stack-

Table 2: Method 2: Neighbor Features and Neural Stacking

Method	AUC	TPR@FPR=0.1
Routed Blend	0.8444	0.2833
Neural Stacking	0.8216	0.2167

ing was used with the goal of the model learning a nonlinear combination of the base model outputs that would be stronger than the simple average ensembling that we have used so far.

Another change in this model is the use of a routed blend, instead of a simple mean ensembling. During validation, in a routed blend, the performance of each individual model as well as a weighted blend of their predictions are compared. The final predictor is the option achieves the best validation AUC. This method is used instead of a simple average of model outputs.

Ultimately, the routed blend performed better than the neural stacked model, with the following results: Routed Blend AUC= 0.8444 TPR@FPR=0.1 = 0.2833 and Neural Stacking Routed Blend AUC= 0.8216 TPR@FPR=0.1 = 0.2167. The Kaggle submission for this model scored AUC = 0.83181.

3.3 Method 3: PC-MMIA and lightweight feature tweaking

Working from method 1, the Lightweight Pipeline, we implemented some tweaks to this model and also incorporated a version of PC-MMIA as set out in the Zheng et al. paper, which we read in milestone 4.

Tweaks made to the lightweight pipeline include adding new pair type ID features: `id_is_seen_image`, `id_is_seen_text`, and `id_pair_type`. These were gleaned from the source id abbreviations, "UI", "SI", "UT", "ST" and made explicit the membership rate for each combination of unseen/seen text and unseen/seen image. In the process, we removed two similar features from the pipeline that were implicitly mapping this relationship. This combination raised the validation AUC score to 0.8482 and kaggle test AUC to 0.83643. We next trained a separate, specialist XGBoost classifier for each possible pair type, which didn't change the AUC score, but led to the discovery that source ids with "mix" were not producing the "1" label, and therefore never members. The membership signal came from "caul-

dron" samples. In response, routing was changed from two groups (cauldron/mix) to three groups (cauldron, mix with SI & UT, mix with UI & ST). We also added a cauldron-specific feature selection capability to the model so that it could select features from cauldron samples rather than the entire dataset, in an effort to refine the membership signal. Changes were also made to the XGBoost classifier, to increase capacity, including raising the number of estimators to 700, lowering the learning rate to 0.02, raising max depth to 4, raising minimum child weight to 3, and reg alpha to 0.5. With these tweaks, the model achieved an AUC of 0.8370. We then increased target encoding smoothing from 10 to 50 and hard-coded the examples with "mix" in their source id to 0, given the discovery that these were always predicted as label 0, however these changes yielded no improvements to the AUC score.

Given existing features were primarily text-based, we added the PC-MMIA features to the model. PC-MMIA is a perturbation-based calibration method that measures the difference in token log-probability before and after image perturbation. The differences are assigned as weights to the token's log-probability, reflecting the assumption that high-weight (high difference) tokens indicate the model saw that image-caption pair in training, and that the token is not just easy to predict. They encompassed a combined image and text signal, features added were:

pcmmia_score: weighted sum of text token log probabilities;

pcmmia_mean_delta: average delta across all text tokens, delta measures drop in token's log-probability when associated image is perturbed;

pcmmia_max_delta: maximum delta across all tokens;

pcmmia_std_delta: standard deviation of delta across all tokens;

pcmmia_top10_delta: mean delta for top 10% tokens with highest memorization signal

pcmmia_top25_delta: mean delta for top 25% tokens with highest memorization signal

These features did not appear in the top features used by the model. This could be due to differences in memorization patterns due to the small size of the fine-tuned SmolVLM-256M model in comparison to the size of models used by Zheng et al. Future work on our model could explore a range of sigma values for varying perturbation. Despite improving validation AUC to 0.8533, the kaggle test AUC decreased to 0.83575.

Table 3: Method 3: PC-MMIA and lightweight feature tweaking

Method	AUC	TPR@FPR=0.1
Pair type ID features with pair-type routing	0.8482	0.2700
+ Cauldron specialist & XG-Boost capacity	0.8485	0.2533
+ PC-MMIA (80 features)	0.8533	0.2681
+ PC-MMIA (50 features, lower corr)	0.8514	0.2612

Our validation set AUC scores were consistently slightly higher than the scores achieved in the kaggle test set. We attempted to address potential overfitting to the validation set by reducing the maximum features selected from 80 to 50 and changing the correlation from 0.995 to 0.98. These changes also yielded no further improvements.

References

- [1] Luis Ibáñez-Lissen, Lorena Gonzalez-Manzano, Jose Maria de Fuentes, Nicolas Anciaux, and Joaquin Garcia-Alfaro. 2025. LUMIA: Linear Probing for Unimodal and MultiModal Membership Inference Attacks Leveraging Internal LLM States. arXiv preprint arXiv:2411.19876.
- [2] Mattern, Justus, Fatemehsadat Mirehghallah, Zhi-jing Jin, Bernhard Schölkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. 2023. Membership Inference Attacks against Language Models via Neighbourhood Comparison. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 11330–11343. <https://arxiv.org/abs/2305.18462>
- [3] Jie Ren, Kangrui Chen, Chen Chen, Vikash Sehwal, Yue Xing, Jiliang Tang, and Lingjuan Lyu. 2025. Self-Comparison for Dataset-Level Membership Inference in Large (Vision-)Language Models. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. ACM.
- [4] Xiaofan Zheng, Huixuan Zhang, and Xiaojun Wan. 2025. Tracing Training Footprints: A Calibration Approach for Membership Inference Attacks Against Multimodal Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17179–17191.

4 Contributions

4.1 Milestone 6

Christina: Literature review, 2 research papers. 0.25

Jennifer: Completed and reported model method 2. 0.25

Rachelle: Completed and reported model method 3. 0.25

Yucai: Completed and reported model method 1. 0.25